

C++ Programming Textbook Notes

Module: Structures in C++ | Evolution from C, Encapsulation & Member Functions

Author: Gaurav Kandpal

Platform: CS-Simplified

Level: Beginner to Advanced

1. STRUCTURES: EVOLUTION FROM C TO C++

What is a Structure?

A **Structure** (defined via the `struct` keyword) is a user-defined compound data type that allows grouping variables of different data types (e.g., `int`, `char[]`, `float`) under a single cohesive unit name.

Limitations of Structures in Standard C

In traditional C programming, structures are strictly passive data storage holders with severe limitations:

- **Variables Only (No Functions):** In C, a structure can hold *only data variables* as members. It cannot contain member functions.
- **No Data Hiding (Lack of Access Specifiers):** In C, all structure members are unconditionally public and globally accessible, preventing encapsulation.
- **Not Treated as Full Built-in Types:** C does not allow standard operators (such as `+` or `-`) to operate directly on user-defined structs (for example, you cannot directly add two complex number structs without dedicated procedural functions).
- **Mandatory 'struct' Tag:** In C, declaring an instance requires repeating `struct book b1;` unless explicit `typedef` aliases are created.

How C++ Expands Structure Capabilities (OOP Evolution)

Because C++ is a superset of C, it retains full backward compatibility with C-style structures while drastically expanding their capabilities to align with **Object-Oriented Programming (OOP)**:

- **Member Functions Inside Structs:** In C++, structures can contain both **data members (variables)** and **member functions (methods)**.
- **Encapsulation Pillar:** By binding state variables and manipulating functions together into a single unit, C++ structures satisfy the OOP principle of **Encapsulation**.
- **Direct Variable Access:** Member functions defined inside a structure have direct, privileged access to all member variables without requiring the dot (`.`) operator!
- **Type Name Promotion:** The `struct` tag becomes a first-class user-defined type name. Writing `book b1;` is fully valid without repeating the `struct` keyword.

2. MEMORY LAYOUT OF STRUCTURES IN RAM

How Struct Memory is Allocated

Defining a structure (the type blueprint) allocates **0 bytes** in physical RAM. When a structure variable (instance) is instantiated, the operating system allocates contiguous memory on the stack to store its member variables sequentially based on their declared sizes and compiler byte alignment padding.

[STRUCT MEMORY LAYOUT IN RAM]

Structure Blueprint (0 Bytes in RAM):
struct book {
 int bookid; // 4 Bytes
 char title[20]; // 20 Bytes
 float price; // 4 Bytes
};

Instantiation: book b1; (Allocates contiguous 28 Bytes in RAM at 0x7ffe00)

bookid (4 Bytes)	title (20 Bytes)	price (4 Bytes)
0x7ffe00	0x7ffe04	0x7ffe18
		0x7ffe1C

3. STRUCTURAL PARADIGM COMPARISON: C VS. C++

Feature	Structures in Standard C	Structures in Modern C++
Allowed Members	Data variables only.	Both data variables and member functions .
Encapsulation	Not supported (passive data container).	Fully supported (bundles state + behavior).
Access Specifiers	None (all members are public).	Supported (public, private, protected; defaults to public).
Variable Instantiation	Requires struct tag: struct book b1;	Direct type name: book b1; (struct is optional).
Member Access	External functions use b1.member.	Member functions access variables directly .

4. STRUCT VS. CLASS IN C++

In C++, a struct and a class are almost identical in capability. Both support member functions, constructors, inheritance, and polymorphism. The **only fundamental difference** is default access visibility:

Construct	Default Member Access	Default Inheritance Visibility
struct	public by default	public inheritance by default
class	private by default	private inheritance by default

5. PRACTICAL CODE DEMONSTRATIONS

Example 1: Procedural C-Style Function Working on Struct

```
// Approach 1: External Function Taking / Returning Struct (C-Style Approach)
#include <iostream>
using namespace std;

struct book {
    int bookid;
    char title[20];
    float price;
};

// Non-member external function returning a struct object
book inputBookData() {
    book b;
    cout << "Enter Book ID, Title, and Price: ";
    cin >> b.bookid >> b.title >> b.price;
    return b;
}

int main() {
    // Instantiating struct variable ('struct' keyword is optional in C++)
    book b1;
    b1 = inputBookData();

    cout << "
Book Details:" << endl;
    cout << "ID: " << b1.bookid << " | Title: " << b1.title << " | Price: $" << b1.price << endl;

    return 0;
}
```

Example 2: Modern C++ Encapsulated Struct with Member Functions

```
// Approach 2: C++ Encapsulated Structure with Internal Member Functions
#include <iostream>
using namespace std;

struct book {
    // Member Variables
    int bookid;
    char title[20];
    float price;

    // Member Function: Directly accesses member variables without dot operator
    void input() {
        cout << "Enter Book ID, Title, and Price: ";
        cin >> bookid >> title >> price;
    }

    // Member Function to display book details
    void display() {
        cout << "Book ID: " << bookid << " | Title: " << title << " | Price: $" << price << endl;
    }
};

int main() {
    book b1;

    // Member functions are invoked on the instance using the dot (.) operator
    b1.input();

    cout << "
Displaying Book Record:" << endl;
    b1.display();

    return 0;
}
```

6. LEGACY CONSOLE FUNCTIONS & STANDARDS EVOLUTION

Legacy Console Functions in <conio.h> (Turbo C++ Era)

In older 16-bit MS-DOS environments (like Turbo C++), students frequently used functions from <conio.h>:

- `clrscr()`: Used to clear the MS-DOS console screen before displaying fresh output.
- `getch()`: Captures a single character input from the keyboard without echoing it to the screen. It was commonly used at the end of `main()` to "hold" the output screen from immediately closing upon execution termination.

Modern Compiler Warning: Deprecation of <conio.h> and <iostream.h>

<conio.h>, `clrscr()`, `getch()`, and <iostream.h> are non-standard legacy artifacts. Modern standard C++ compilers (GCC, Clang, MSVC) do not support them. Modern IDEs keep console output visible automatically upon program completion.

Best Practice: When to Use struct vs. class

Use `struct` for plain-old-data (POD) containers where all fields are inherently public (e.g., 2D Point coordinates `{int x, y;}`, RGB color triplets). Use `class` whenever you require data hiding (`private` fields) and invariant validation!